RECLAIM — Implementation Plan (Founder's Code 2026)

> **Read this whole file before writing any code.** It is the single source of truth for the Reclaim prototype. It is written so that any developer or AI coding agent can pick up the project at any point, see what is done (Section 18 task board), and continue.
>
> Event: Founder's Code 2026, 24-hour hackathon, SMIT Chennai, 14–15 Oct 2026. Domain: Cybersecurity & Digital Safety (example category: "fraud-reporting assistance"). Owner: Shriram (tech lead). One business-track teammate handles pitch/market.

---

0. TL;DR for an AI agent picking this up

1. Product: **Reclaim** — a mobile-first web app that walks a UPI/bank-fraud victim through the three official reports (1930 call → cybercrime.gov.in complaint → written bank report), generates every text they need, and tracks the **RBI 5-day compensation window** and the **bank's 45-day resolution deadline**. A **bank dashboard** shows incoming cases live.
2. Stack: **React + Vite + TypeScript + Tailwind** (frontend), **FastAPI + SQLite** (backend), **Gemini Flash (free tier)** for screenshot extraction + scam tagging, deterministic fallbacks.
3. The demo must **never fail**. Every AI call has a cached/demo fallback (Section 9).
4. Time is controlled by a **demo clock** (`DEMO_NOW`), because the RBI rule only applies to transactions on/after **1 Jan 2027** and the event is in Oct 2026 (Section 7.1).
5. Build order = Section 17 timeline. Current status = Section 18 task board. Update it as you go.
6. Never use real personal/financial data. Only the fake samples in `samples/` .

---

1. The story (what the demo must make the jury feel)

Ravi, 34, Chennai, gets a call: "Your KYC has expired, your account will be blocked." He taps the link, enters his UPI PIN. **₹18,000** is gone (Google Pay → `kyc.update@ybl` , 12 Jan 2027, 7:42 PM).

Without Reclaim: he panics, doesn't know about 1930, can't find the UTR, never writes to his bank. Day 6 he files a messy complaint. Money gone. He also **misses RBI compensation** because compensation needs reports to **both** 1930/NCRP **and** the bank within **5 calendar days**.

With Reclaim: his bank's SMS has a Reclaim link. In ~1 hour he has filed all three reports correctly. Reclaim shows **"Compensation-eligible: ₹15,300"** (85% of ₹18,000). His bank's fraud team sees his complete case appear on their dashboard instantly.

Closing line of the pitch: *"With Reclaim, Ravi gets ₹15,300 back. Without it, ₹0."*

---

2. Verified facts the product relies on (cite these, don't invent others)

Fact	Source (secondary — read the primary RBI notification before pitching)
RBI (Commercial Banks – Responsible Business Conduct) Third Amendment Directions, 2026 apply to electronic banking transactions on/after **1 Jan 2027**	ELP Law summary (Jun 2026)
Small-value fraud compensation: losses **up to ₹50,000**, pays **min(85% of net loss, ₹25,000)**, **once per lifetime**, individuals incl. sole proprietors	ELP Law / corplawupdates summaries
Compensation condition: report to **bank AND NCRP/1930 within 5 calendar days**	same
Banks must resolve domestic fraud complaints within **45 calendar days**	ELP Law summary

Fact	Source (secondary — read the primary RBI notification before pitching)
1930 call → victim gets a **14-digit acknowledgement number** by SMS; must then complete the complaint on NCRP (cybercrime.gov.in)	the420.in explainer
NCRP financial-fraud form needs: bank/wallet name, **transaction ID / 12-digit UTR**, date, amount; description **≥ 200 characters**, avoid special chars `# $ @ ^ *`; ID proof JPG/JPEG/PNG **≤ 5 MB**; evidence files **≤ 10 MB**	RBL Bank guide
~**51%** of UPI-fraud victims filed no official complaint (LocalCircles, 32,000+ responses, Jun 2025)	Business Standard
Odisha lost ~**₹370 crore** (2023–25), recovered ~**2.7%**; police cite delayed reporting	the420.in
Free-tier Gemini data may be used by Google / reviewed by humans → **never send real personal data**	Gemini pricing page summary (agentdeals.dev)

Unverified — say "to be confirmed" if asked: exact RBI Ombudsman escalation timing (commonly 30 days of no reply), Ombudsman portal URL (`cms.rbi.org.in`), whether cooperative banks are covered (directions name *commercial* banks — pitch to "banks").

---

3. Scope

3.1 In scope (prototype)

- Victim web app (mobile-first), 9 screens (Section 6).
- Screenshot → fields extraction (Gemini) + manual entry.
- Scam-type classification with confidence.
- Eligibility engine + countdown (5-day) + 45-day bank deadline.
- Generated texts: 1930 call script, NCRP complaint description, bank email/letter.
- Case timeline (Fraud → 1930 → Portal → Bank) that turns green.
- Case pack PDF download.
- Bank dashboard (live list of cases, deadline status, scam-type breakdown).
- **Demo Stage** page: phone frame + dashboard side by side for the projector.
- English + Tamil UI toggle (key screens).

3.2 Out of scope (say "roadmap" if asked)

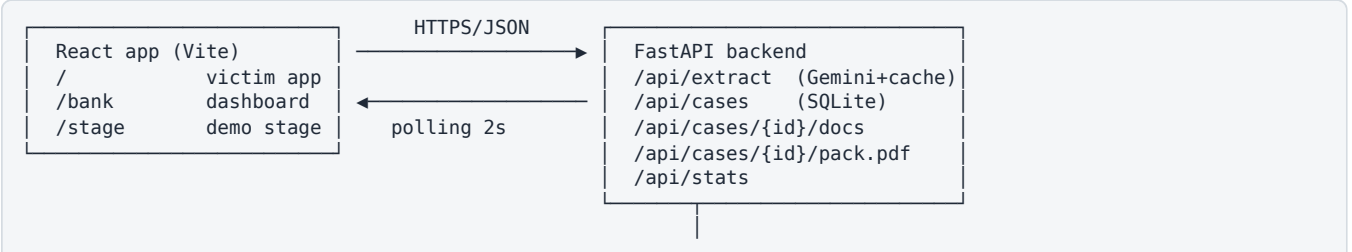
- Auto-filing on NCRP (no public API — by design the victim submits).
- Real bank integrations, real SMS/WhatsApp reminders (simulate in UI), auth/login for victims, payments, multi-tenant bank accounts.

3.3 Non-negotiable product rules

- Never ask for OTP, PIN, password, card number, full account number.
- Never file on the user's behalf; never promise refunds.
- Never charge or handle victims' money. Show "recovery scam" warning on final screens.
- Every extracted field is **user-confirmed** before documents are generated.

---

4. Architecture





- Single backend process, single SQLite file. No queues, no websockets (polling is enough and fails less).
- Frontend calls backend via `VITE_API_URL`. In dev, Vite proxy `/api` → `localhost:8000`.

---

5. Repository structure

```
reclaim/
├── README.md                      # how to run, demo script link
├── RECLAIM_IMPLEMENTATION_PLAN.md # this file
├── PRE_EXISTING.md                # declare anything not built during the event (should be empty)
├── .gitignore
├── backend/
│   ├── requirements.txt
│   ├── .env.example
│   ├── app/
│   │   ├── main.py                # FastAPI app, CORS, routers
│   │   ├── config.py              # env vars, DEMO_NOW handling
│   │   ├── db.py                  # SQLModel engine/session
│   │   ├── models.py              # Case table
│   │   ├── schemas.py             # Pydantic request/response models
│   │   ├── clock.py               # now() -> demo-aware datetime
│   │   ├── eligibility.py         # RBI rules engine (pure functions + tests)
│   │   ├── extract/
│   │   │   ├── gemini.py          # Gemini call + JSON schema
│   │   │   ├── validate.py        # regex validation/normalisation
│   │   │   └── cache.py           # sha256(image) -> fixture json
│   │   ├── classify.py            # scam-type from description (Gemini + keyword fallback)
│   │   ├── docs_gen.py            # 1930 script, NCRP text, bank letter (templates)
│   │   ├── pdf_pack.py            # ReportLab case pack
│   │   ├── routers/
│   │   │   ├── extract.py
│   │   │   ├── cases.py
│   │   │   └── stats.py
│   │   └── seed.py                # seeds dashboard with demo cases
│   ├── fixtures/                  # cached extraction results for sample images
│   │   ├── gpay_18000.json
│   │   ├── phonepe_9500.json
│   │   ├── paytm_42000.json
│   │   └── sms_indianbank_18000.json
│   └── tests/
│       ├── test_eligibility.py
│       ├── test_validate.py
│       └── test_docs_gen.py
├── frontend/
│   ├── package.json
│   ├── .env.example
│   ├── index.html
│   ├── tailwind.config.js
│   ├── src/
│   │   ├── main.tsx
│   │   ├── App.tsx                # routes
│   │   ├── api.ts                 # typed fetch wrappers
│   │   ├── store.ts               # Zustand case store (persist caseId in localStorage)
│   │   ├── i18n/{en.ts,ta.ts,index.ts}
│   │   ├── lib/{format.ts,countdown.ts}
│   │   ├── components/            # Button, Card, Field, Timeline, Countdown, Pill, PhoneFrame, CopyBox
│   │   ├── screens/               # S1Welcome … S9Done (Section 6)
│   │   ├── bank/Dashboard.tsx
│   │   └── stage/Stage.tsx
│   └── public/samples/            # same sample images, for one-tap demo upload
├── samples/                       # fake screenshots (PNG) + generator script
│   └── make_samples.py
└── pitch/
    ├── deck_outline.md
    └── demo_script.md
```

---

6. Screen specifications (victim app)

Global layout: max width 420px, centred; on desktop the Demo Stage wraps it in a phone frame. Header: small Reclaim logo (shield icon + "Reclaim") left, language toggle `EN | தமிழ்` right. Every screen has one primary button at the bottom. Back arrow top-left (except S1).

S1 — Welcome (`/`)

- Icon: shield-check. Title: **"Lost money to a scam?"**
- Sub: "We'll help you report it correctly in the next 10 minutes."
- Question card: **"Is the scam still happening right now?"** → buttons `Yes, right now` / `No, it's over`.
- `Yes` → **S1b Emergency**.
- `No` → choice: `I lost money` (→ S2) / `Something else happened` (→ S1c).
- Footer note (always visible): "Always free. We never ask for money, OTPs or passwords."

S1b — Emergency (`/emergency`)

- Red banner. Bullets: "Hang up now." "Police never arrest anyone on a video call." "Don't send more money." "Don't share OTP or PIN."
- Buttons: `Call 1930` (`tel:1930`) and `I'm safe now — continue` (→ S2).

S1c — Non-financial (`/other`)

- Short guide: report on cybercrime.gov.in under the relevant category; link button. Exit.

S2 — Upload proof (`/upload`)

- Title "Upload your proof". Sub "Payment screenshot or bank SMS".
- Drop zone / file picker (accept image/*). On mobile allow camera.
- **"Use a sample"** row (demo helper, only when `VITE_DEMO_MODE=true`): 4 thumbnails from `public/samples/` . Tapping uploads that file exactly like a real upload.
- Link: "I don't have a screenshot → enter details manually" (→ S3 with empty form).
- Help accordion "Where do I find it?": GPay/PhonePe/Paytm → History → tap payment → screenshot.
- "What happened?" textarea + mic button (Web Speech API, `lang = en-IN` or `ta-IN`). Placeholder: "Someone called saying my KYC expired and sent a link…"
- Primary: `Continue` . On submit: show skeleton "Reading your screenshot…" (min 1.2 s for drama, max until response) → POST `/api/extract` → S3.
- Errors: no file and no manual choice → inline "Add a screenshot or enter details manually".

S3 — Check details (`/confirm`)

- Title "Check these details". Editable fields, each with a small "from screenshot" badge:
- Amount (₹, integer), UTR / reference (12 digits), Date & time, Your bank, Payment app, Payment method (UPI / Card / Net banking), Scammer UPI ID / account (masked display allowed).
- Low-confidence fields (`confidence < 0.7`) get amber outline + "Please check".
- "Add another transaction" (adds a second card; totals summed). Max 5.
- Scam type card: `Fake KYC update` + pill `92% match` . If confidence < 0.6 → show chip list to pick: KYC, Job/task, Investment, Digital arrest, Fake customer care, Sextortion, Loan app, Other.
- Question: "Have you received RBI fraud compensation before?" `No` (default) / `Yes` .
- Primary `All correct` → POST `/api/cases` → S4. Validation: UTR 12 digits (warn, don't block, if card/netbanking ref differs), amount > 0, date not in future (demo clock).

S4 — Your plan + countdown (`/case/:id`)

- Big countdown (D h m, ticking every second) — "Time left to stay eligible for compensation".
- Pill: eligibility result (Section 7). Examples:

- green "Eligible for up to ₹15,300"
- amber "Not covered by small-value compensation — report now for the best chance of a hold"
- grey "Compensation window has closed — still report"
- Timeline component: `Fraud ✓ → 1930 ○ → Portal ○ → Bank ○`.
- Card "You need 3 reports. RBI compensation needs all of them within 5 days."
- Primary `Start step 1: call 1930`.
- This is the **case home**: every later step returns here; done steps show green with their reference number.

S5 — Step 1: Call 1930 (`/case/:id/1930`)

- CopyBox "Read this to the officer" with generated **1930 script** (Section 8.1).
- Button `Call 1930` (`tel:1930`).
- Input "14-digit acknowledgement number from the SMS" (validate `^\d{14}$` ; soft-warn if not starting with `3`). Save → PATCH case `ack_1930`.
- Secondary links: "Line busy? File on the portal first" (→ S6, step stays pending); "Didn't get the SMS?" → tip + `Mark as called, number pending` (status `called_pending`).

S6 — Step 2: Portal complaint (`/case/:id/portal`)

- CopyBox "Complaint description" with generated **NCRP text** (Section 8.2), char counter (must show ≥ 200), `Copy` button (toast "Copied").
- Checklist card: Category "Online Financial Fraud", Bank, UTR, Amount, Date, 1930 ack no., attachments (screenshot ≤ 10 MB, ID proof JPG/PNG ≤ 5 MB).
- Button `Open cybercrime.gov.in` (new tab).
- Input "Portal acknowledgement number" (free text, required to mark done). Save → `ack_portal`.
- Link "Portal not working? Save and remind me" (simulated reminder toast).

S7 — Step 3: Bank report (`/case/:id/bank`)

- CopyBox email with generated **bank letter** (Section 8.3), `Copy` + `Open email` (`mailto:` with subject/body pre-filled; bank fraud email from `banks.json` , else blank).
- Tip: "No email? Use your bank app's 'Report fraud', call customer care, or print this at your branch."
- Input "Bank complaint / ticket number". Save → `bank_ref` , `bank_reported_at`.
- On save the case appears/updates on the **bank dashboard** (it already exists from S3; status changes).

S8 — Done (`/case/:id/done`) — reached automatically when all 3 refs saved

- Success icon. "All 3 reports done." Pill "Compensation-eligible: ₹15,300" (or the right eligibility message). Sub "85% of ₹18,000 · done in 1 hr 12 min" (completed_at − fraud_at).
- Card: "Bank must resolve by 26 Feb 2027" (fraud/report date + 45 days), "Next reminder: Day 30".
- Button `Download case pack (PDF)` → `/api/cases/:id/pack.pdf`.
- Warning card: "Anyone offering to 'recover' your money for a fee is a scammer. Never pay."

S9 — Follow-up (shown inside S4 after done; demo via "Skip to day 30" in Stage controls)

- Day 30: "Has your bank replied?" `Yes, resolved` / `No reply` → generates a reminder letter.
- Day 45: if unresolved → "Draft RBI Ombudsman complaint" (text only; mark as to-be-verified).

Copy rules

Sentence case, short, no exclamation marks, no "please". Indian number format (`₹18,000` , `₹1.5 lakh`). Dates `12 Jan 2027, 7:42 PM` (Asia/Kolkata).

---

7. Eligibility engine (`backend/app/eligibility.py`)

7.1 Demo clock

- `config.DEMO_NOW` (ISO string, optional). If set, `clock.now()` returns `DEMO_NOW + (real_now − server_start_time)` so the countdown still ticks.
- Default demo value: `2027-01-12T20:01:00+05:30` (19 minutes after Ravi's fraud).
- Stage page has controls to jump time: `+1 day`, `Jump to day 4 23h`, `Jump to day 30`, `Reset` → POST `/api/demo/clock` (only when `DEMO_MODE=true`).

7.2 Rules (constants in one place)

```
RULE_EFFECTIVE_FROM = date(2027, 1, 1)      # transactions on/after
COMP_MAX_LOSS       = 50_000                # ₹, loss cap for scheme
COMP_RATE           = 0.85
COMP_CAP            = 25_000                # ₹
REPORT_WINDOW_DAYS  = 5                     # calendar days, both bank and NCRP/1930
BANK_RESOLVE_DAYS   = 45                    # domestic
```

7.3 Function contract

```
def evaluate(fraud_at: datetime, total_loss: int, prior_compensation: bool,
             ack_1930: str|None, ack_portal: str|None, bank_reported_at: datetime|None,
             now: datetime) -> Eligibility
```

`Eligibility` fields: `status` (one of below), `max_compensation` (int ₹), `deadline_at` (fraud_at + 5 days), `seconds_left` (≥0), `bank_resolve_by` (bank_reported_at or fraud_at + 45 d), `message_key` (i18n key), `reasons` (list of strings).

Status logic (first match wins):

1. `fraud_at.date() < RULE_EFFECTIVE_FROM` → `NOT_APPLICABLE_DATE`
2. `prior_compensation` → `NOT_APPLICABLE_PRIOR`
3. `total_loss > COMP_MAX_LOSS` → `NOT_APPLICABLE_AMOUNT`
4. `now > deadline_at` and not both reported before deadline → `WINDOW_MISSED`
5. NCRP reported (`ack_1930` or `ack_portal`) **and** bank reported, both before deadline → `ELIGIBLE_COMPLETE`
6. else → `ELIGIBLE_PENDING`

`max_compensation = min(round(COMP_RATE * total_loss), COMP_CAP)` for statuses 5–6, else 0. Ravi: min(15,300, 25,000) = **15,300**.

Note in UI: "Estimate. Your bank decides final eligibility." (Reclaim never judges negligence.)

7.4 Unit tests (must pass)

Case	Expected
₹18,000, 12 Jan 2027, now +19 min, nothing reported	ELIGIBLE_PENDING, 15,300
same, all reported day 1	ELIGIBLE_COMPLETE, 15,300
₹40,000	25,000 cap
₹60,000	NOT_APPLICABLE_AMOUNT, 0
fraud 20 Dec 2026	NOT_APPLICABLE_DATE
now = fraud + 6 days, bank not reported	WINDOW_MISSED
prior_compensation true	NOT_APPLICABLE_PRIOR

8. Generated documents (`backend/app/docs_gen.py`)

Plain string templates (Python f-strings). **No LLM needed** → deterministic and safe. Optional: LLM polishes the user's own description into 2–3 clean sentences (`incident_summary`); fallback = user text as-is. Strip `# $ @ ^ *` from NCRP text.

8.1 1930 call script

```
I want to report a cyber financial fraud.
I lost ₹{amount} by {method} on {date} at {time}.
Transaction reference (UTR): {utr}.
The money went to {scammer_id}.
My bank is {bank}. My mobile number is the one I'm calling from.
It happened like this: {incident_summary_one_line}
Please give me the acknowledgement number.
```

8.2 NCRP complaint description (must be ≥ 200 chars; pad with structured facts if short)

```
On {date} at {time}, I was the victim of a {scam_type_label} fraud. {incident_summary}
As a result, ₹{amount} was debited from my {bank} account via {method}
(transaction reference / UTR {utr}) and sent to {scammer_id}.
I reported this to the 1930 helpline (acknowledgement number {ack_1930 or "pending"}).
I request that the amount be traced and put on hold, and that action be taken against the
beneficiary account. Screenshots of the transaction are attached.
```

8.3 Bank letter / email

```
Subject: Unauthorised {method} debit of ₹{amount} — UTR {utr} — request for hold and compensation

To the Fraud / Grievance Desk, {bank},

I am reporting a fraudulent transaction on my account.
- Amount: ₹{amount}
- Date and time: {date}, {time}
- Reference / UTR: {utr}
- Beneficiary: {scammer_id}
- Reported to 1930: acknowledgement {ack_1930}
- Reported on cybercrime.gov.in: acknowledgement {ack_portal}

Summary: {incident_summary}

I request you to (1) place an immediate hold / freeze on the beneficiary funds where possible,
(2) register this complaint and share a reference number, and (3) process my case under the
RBI (Commercial Banks – Responsible Business Conduct) Third Amendment Directions, 2026,
including the applicable compensation for small-value frauds.

Name: {name or "[Your name]"}
Account (last 4 digits): {acct_last4 or "[XXXX]"}
Registered mobile: {mobile or "[Your mobile]"}
Date: {today}
```

8.4 Day-30 reminder and Day-45 Ombudsman draft

Short templates referencing `bank_ref` and dates. Mark Ombudsman text "Draft — verify current RBI Ombudsman process before submitting".

---

9. AI extraction + classification (never let it break the demo)

9.1 Order of attempts in `POST /api/extract`

1. **Cache:** `sha256(image_bytes)` → `backend/fixtures/{hash}.json` (also map by sample filename). If hit → return immediately with `source: "cache"` (still show 1.2 s loader in UI).
2. **Gemini** (timeout 12 s): image + prompt below, `response_mime_type="application/json"` with schema. On success → validate (9.3) → return `source: "ai"` .

3. **Fallback:** return empty fields with `source: "manual"` and a friendly message "We couldn't read it — please type the details." (Optional: Tesseract + regex.)

9.2 Gemini prompt (extraction + classification in one call)

Model: a current **free-tier Gemini Flash** model (put the exact model ID from Google AI Studio in `GEMINI_MODEL`; do not hard-code). SDK: `google-genai`.

```
You read Indian payment screenshots and bank SMS messages.
Return ONLY JSON matching the schema. Use null when a value is not visible. Never guess digits.
Fields:
- amount_inr: integer rupees debited (no symbols, no paise)
- utr: the UPI transaction ID / UTR / reference number exactly as shown (digits only if UPI)
- txn_datetime: ISO 8601 with +05:30 if date and time are visible
- payer_bank: the payer's bank name if visible
- payment_app: Google Pay | PhonePe | Paytm | BHIM | Bank app | SMS | Other
- method: UPI | Card | NetBanking | Other
- payee_id: the receiver UPI ID, account, or name exactly as shown
- field_confidence: object with a 0..1 confidence for each field above

Also classify the user's description into one scam_type from:
fake_kyc, job_task, investment, digital_arrest, fake_customer_care, sextortion, loan_app, other
and give scam_confidence 0..1 and a one-sentence incident_summary in plain English.

User description: """{description}"""
```

9.3 Validation/normalisation (`extract/validate.py`)

- UTR: keep digits; UPI UTR should be **12 digits** → else confidence ≤ 0.5.
- Amount: int > 0; if > 10,00,000 → confidence ≤ 0.5.
- Datetime: parse; if missing time → keep date, confidence 0.6.
- UPI ID regex: `^[\w.\-]{2,256}@[a-zA-Z]{2,64}$`.
- Never return account numbers in full: mask all but last 4 for display.

9.4 Keyword fallback classifier (`classify.py`)

Map keywords → type: `kyc|pan|aadhaar update` → fake_kyc; `task|like|youtube|part time|telegram` → job_task; `stock|trading|crypto|ipo` → investment; `cbi|police|customs|arrest|parcel|narcotics` → digital_arrest; `customer care|refund|helpline` → fake_customer_care; `video call|nude|morph` → sextortion; `loan|app|instant loan` → loan_app. Confidence 0.6 if matched, else other/0.3.

9.5 Fixtures

For every sample image in `samples/`, store the **expected** extraction JSON in `backend/fixtures/`. These are both the cache and the test oracle.

---

10. Backend API spec (FastAPI, prefix `/api`)

Data model `Case` (SQLModel)

```
id: str (uuid4 short, e.g. "RC-7F3K2")          status: str  # draft|in_progress|complete|missed
created_at, updated_at: datetime                fraud_at: datetime
amount_total: int                               transactions: JSON (list of txn dicts)
bank: str                                       payment_app: str
method: str                                     payee_id: str
scam_type: str                                  scam_confidence: float
incident_summary: str                           description_raw: str
prior_compensation: bool                        language: str ("en"|"ta")
ack_1930: str|None      ack_1930_at: datetime|None   called_pending: bool
ack_portal: str|None    ack_portal_at: datetime|None
bank_ref: str|None      bank_reported_at: datetime|None
completed_at: datetime|None
followup_state: str|None   # none|day30_no_reply|resolved|ombudsman_drafted
```

```
source: str              # cache|ai|manual
is_demo_seed: bool
```

Endpoints

Method	Path	Body / params	Returns
GET	`/api/health`	–	`{ok, demo_mode, now}`
POST	`/api/extract`	multipart: `image` (optional), `description`, `language`	`{fields{...}, field_confidence{...}, scam_type, scam_confidence, incident_summary, source}`
POST	`/api/cases`	confirmed fields + `prior_compensation`, `language`	full `CaseOut` (incl. eligibility)
GET	`/api/cases/{id}`	–	`CaseOut` (eligibility recomputed with `clock.now()`)
PATCH	`/api/cases/{id}`	any of `ack_1930`, `called_pending`, `ack_portal`, `bank_ref`, `followup_state`	`CaseOut` (sets `*_at` timestamps; sets `completed_at` + status when all 3 done)
GET	`/api/cases/{id}/docs`	–	`{script_1930, ncrp_text, ncrp_chars, bank_subject, bank_body, reminder_day30, ombudsman_draft}`
GET	`/api/cases/{id}/pack.pdf`	–	PDF file
GET	`/api/cases`	`?limit=50`	list of `CaseRow` for dashboard (newest first)
GET	`/api/stats`	–	`{total, eligible_amount_sum, by_scam_type{}, due_in_24h, missed, completed_rate}`
POST	`/api/demo/clock`	`{action: "reset"|"plus_day"|"jump_day4"|"jump_day30"}`	`{now}` (DEMO_MODE only)
POST	`/api/demo/reset`	–	wipes non-seed cases, reseeds (DEMO_MODE only)

`CaseOut` = all fields + `eligibility{status, max_compensation, deadline_at, seconds_left, bank_resolve_by, message_key}` + `steps{1930, portal, bank}` each `{done: bool, ref, at}`.

CORS: allow frontend origin(s) from env. Max upload 10 MB; accept `image/png`, `image/jpeg`, `image/webp`.

---

11. Bank dashboard (`/bank`)

Purpose: shows the jury **who pays** and why. Polls `/api/cases` + `/api/stats` every 2 s.

Layout (desktop, 1280px):

- Header: "Reclaim for banks · Indian Bank (demo)" + live dot "Live".
- KPI row (4 tiles): Open cases · Due in < 24 h (amber) · Missed deadlines (red) · Compensation exposure ₹ (sum of `max_compensation` for eligible).
- Table: Case ID · Customer (masked "R••• K." / last 4 mobile) · Amount · Scam type pill · Reported (1930 ✓ Portal ✓ Bank ✓ icons) · Deadline (countdown, colour: green >48 h, amber <24 h, red missed) · Status.
- New case row flashes highlight for 3 s when it first appears (this is the demo moment).
- Right panel: "Scam types this week" horizontal bars (simple divs, no chart lib needed).

- Row click → side drawer with full case, docs, `Download case pack`.
- Seed data (`seed.py`): ~12 realistic fake cases across scam types and deadline states so the dashboard never looks empty. Ravi's live case appears on top.

---

12. Demo Stage (`/stage`) — the projector view

- Left: `PhoneFrame` (375×780) containing an `<iframe src="/">` (victim app).
- Right: `<iframe src="/bank">` (dashboard).
- Bottom bar (small, only DEMO_MODE): demo clock display + buttons `Reset demo` , `+1 day` , `Jump to day 4 23h` , `Jump to day 30` .
- This lets one laptop on the projector show **both sides at once**: as Ravi saves the bank reference on the left, his case turns green on the right.

---

13. Case pack PDF (`pdf_pack.py` , ReportLab)

One–two A4 pages: Reclaim header · case ID · generated date · transaction table · scam type · timeline with timestamps and all reference numbers · the three generated texts · deadlines · "Keep this document. Never pay anyone to recover money." Font: a TTF with ₹ glyph (e.g. Carlito/Noto Sans). Test that ₹ renders.

---

14. Design system

- Feel: calm, trustworthy, banking-grade. Lots of white space, one accent.
- Colours (Tailwind theme extend):
- `navy` #0F2A44 (headers, primary text accents)
- `teal` #0E7C86 (primary buttons, links, success-adjacent accent)
- `danger` #B42318 (countdown, emergency) · `warn` #B54708 · `ok` #067647
- neutrals: slate scale
- Font: Inter (Google Fonts) for UI; Noto Sans Tamil for Tamil.
- Radius 12px cards, 10px buttons; 0.5–1px borders; no heavy shadows.
- Icons: `lucide-react` (ShieldCheck, Upload, Phone, FileText, Mail, Check, Clock, AlertTriangle, Download).
- Countdown: tabular numbers, 40px, red when < 24 h left.
- Motion: only 150–250 ms fades/slides; timeline step turns green with a 300 ms scale pop.
- Accessibility: tap targets ≥ 44px, contrast AA, labels on all inputs.

---

15. i18n (English + Tamil)

- `src/i18n/en.ts` and `ta.ts` export the same keys; `t(key)` helper; toggle stored in store.
- Translate at minimum: S1, S1b, S2, S4, S8 headings/buttons, eligibility messages.
- Generated official documents stay in **English** (portal/bank expect it); show a Tamil "what this says" line above each CopyBox if time allows.
- Have a Tamil speaker check strings. Do not machine-translate legal text silently.

---

16. Environment & setup

backend/.env.example

```
GEMINI_API_KEY=
GEMINI_MODEL=           # exact free-tier Flash model id from Google AI Studio
DEMO_MODE=true
```

```
DEMO_NOW=2027-01-12T20:01:00+05:30
CORS_ORIGINS=http://localhost:5173
DB_PATH=./reclaim.db
```

frontend/.env.example

```
VITE_API_URL=http://localhost:8000
VITE_DEMO_MODE=true
```

Commands

```
# backend
cd backend && python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install fastapi uvicorn[standard] sqlmodel python-multipart google-genai reportlab pydantic-settings pytest
uvicorn app.main:app --reload --port 8000
pytest -q

# frontend
cd frontend && npm create vite@latest . -- --template react-ts
npm i react-router-dom zustand lucide-react clsx
npm i -D tailwindcss postcss autoprefixer && npx tailwindcss init -p
npm run dev
```

(Use the Tailwind setup that matches the installed major version — check its docs if `init` differs.)

Phone demo

- Same Wi-Fi: run Vite with `--host`, open `http://<laptop-ip>:5173` on phone.
- Or `ngrok http 5173` (and proxy `/api` through Vite so only one tunnel is needed).
- Venue Wi-Fi may be bad → carry a phone hotspot; everything else runs locally.

---

17. 24-hour timeline (mapped to jury checkpoints)

Time	Goal	Tasks (IDs from Section 18)	Must be demoable
11:00–11:30	Repo + skeletons	T01–T04	`GET /api/health` ok, Vite app renders
11:30	Jury topic validation	—	1-page proposal + 30-s pitch
11:30–14:30	Upload → extract → confirm	T05–T11	**2:30 jury:** sample screenshot → fields + scam type on screen
14:30–19:00	Case, eligibility, countdown, Steps 1–3, docs	T12–T20	Full victim flow S1→S8 on sample
19:00	Challenge 1	Section 20	Respond within ~2 h
19:00–00:00	Bank dashboard, seed, Stage page, PDF	T21–T27	Both sides live on Stage
00:00	Challenge 2	Section 20	
00:00–06:00	Tamil, edge cases, polish, deploy, tests	T28–T35 (sleep in 2 shifts)	Deployed URL works on phone
06:00	Challenge 3	Section 20	
06:00–08:00	Bug fixes only after 08:00 (**feature freeze**)	T36–T38	
08:00–09:00	Rehearse demo ×3, record backup video	T39–T40	
09:00	Final jury evaluation	—	Live demo (Section 21)
10:00–11:00	Submission window	T41	Repo, deck, video submitted
11:00–12:30	Final presentations	—	3-min pitch + Q&A

Git: commit every 30–60 min, messages like `feat(extract): gemini call with json schema`, `fix(eligibility): window uses calendar days`. Tag `v-jury-1430`, `v-final`.

---

18. Task board (update status as you work)

Status: ☐ todo · ◖ doing · ☑ done. Each task has acceptance criteria (AC).

Setup

- ☐ T01 Create repo, folders per Section 5, `.gitignore`, `PRE_EXISTING.md`. AC: pushed to GitHub.
- ☐ T02 Backend skeleton: config, clock, db, `/api/health`. AC: returns demo `now`.
- ☐ T03 Frontend skeleton: Vite+TS+Tailwind+Router, routes stubbed, design tokens. AC: `/`, `/bank`, `/stage` render.
- ☐ T04 Sample images (`samples/make_samples.py` or hand-made) + copy to `frontend/public/samples/`. AC: 4 PNGs (GPay ₹18,000; PhonePe ₹9,500; Paytm ₹42,000; Indian Bank SMS ₹18,000), all fake data.

Extraction

- ☐ T05 Fixtures JSON for the 4 samples. AC: valid against schema.
- ☐ T06 `/api/extract` cache path (hash + filename). AC: sample returns fixture in < 100 ms.
- ☐ T07 Gemini call with JSON schema + 12 s timeout. AC: works on 2 samples with key set; errors fall back.
- ☐ T08 Validation/normalisation + masking. AC: `test_validate.py` passes.
- ☐ T09 Keyword fallback classifier. AC: digital-arrest text → `digital_arrest`.
- ☐ T10 S2 Upload screen (file, samples row, description, mic). AC: submit shows loader then navigates.
- ☐ T11 S3 Confirm screen (editable fields, confidence styling, scam chip, prior comp). AC: edits persist into POST.

Case flow

- ☐ T12 Case model + `POST/GET/PATCH /api/cases`. AC: create → fetch → patch ack.
- ☐ T13 Eligibility engine + tests (Section 7.4). AC: all pass.
- ☐ T14 S4 case home: countdown, eligibility pill, timeline. AC: ticks each second; matches backend `seconds_left`.
- ☐ T15 `docs_gen.py` + `/docs` endpoint + tests. AC: NCRP text ≥ 200 chars, no `#$@^*`.
- ☐ T16 CopyBox component (copy + toast + char counter).
- ☐ T17 S5 1930 step (script, tel link, 14-digit input, pending option).
- ☐ T18 S6 Portal step (text, checklist, open site, ack input).
- ☐ T19 S7 Bank step (letter, mailto, bank ref input).
- ☐ T20 S8 Done screen + auto-redirect when 3 steps done. AC: shows ₹15,300 for Ravi.

Bank side & demo

- ☐ T21 `seed.py` with ~12 cases + `/api/demo/reset`. AC: dashboard never empty.
- ☐ T22 `/api/cases` list + `/api/stats`.
- ☐ T23 Dashboard KPI tiles + table + polling 2 s.
- ☐ T24 New-row highlight + deadline colours + scam-type bars.
- ☐ T25 Case drawer with docs + PDF button.
- ☐ T26 Stage page (phone frame + dashboard iframes + demo clock controls).
- ☐ T27 Case pack PDF with ₹ rendering.

Hardening & polish

- ☐ T28 S1/S1b/S1c safety branches.
- ☐ T29 Multiple transactions on S3 (sum).
- ☐ T30 Eligibility edge messages (> ₹50k, missed window, before 2027, prior comp).
- ☐ T31 Tamil strings for key screens + toggle.

- ☐ T32 Day-30 / Day-45 follow-up (S9) via demo clock.
- ☐ T33 Empty/error states everywhere (network error toast + retry).
- ☐ T34 Deploy (Vercel + Render) or ngrok; test on a real phone.
- ☐ T35 README with run steps + screenshots.
- ☐ T36 Full regression: run Section 22 checklist.
- ☐ T37 Lighthouse/mobile check: no layout break at 360px width.
- ☐ T38 Remove debug logs; ensure no secrets committed.

Pitch

- ☐ T39 Deck (Section 21) finalised.
- ☐ T40 Backup demo video (60–90 s, screen recording of Stage).
- ☐ T41 Submission: repo link, deck, video, proposal.

---

19. Business model (for slides and Q&A — figures are estimates, say so)

Users (free, always): fraud victims.

Payers:

Stream	Price (estimate)	Why they pay
Bank licence (white-label)	₹6 lakh / bank / year	Complete, ordered claims; meet 45-day resolution; audit trail under new RBI rules
Cyber-insurance claim packs	₹150 / verified claim	Clean timestamped evidence → faster claim settlement
Fintech / UPI app integration	₹2 lakh / year + usage	"Report fraud" button opens Reclaim in-app

- **Illustration:** 20 banks × ₹6 lakh = ₹1.2 crore/year before insurer revenue.
- **Cost (estimate):** cloud + LLM + messaging ≈ low lakhs/year at pilot scale.
- **GTM:** "Victims don't find Reclaim — their bank sends it to them." Pilot with one mid-size bank before the Jan 2027 rule → 1930/cyber-cell SMS links + branch QR posters → insurers → more banks.
- **Moat:** rules engine + evidence pipeline + bank workflow; distribution through banks.

---

20. Surprise-challenge playbook (7 PM, 12 AM, 6 AM)

Likely asks and the fastest good answer:

Challenge	Response (target ≤ 2 h)
Regional language / accessibility	Tamil toggle (T31) + voice input on S2 + larger text mode
Privacy / data protection	Mask fields, "Delete my case" button (DELETE endpoint), consent checkbox on S2, local-only mode note
Offline / low network	Save draft to localStorage; manual entry path; SMS-style text summary
Scale / analytics	Dashboard stats: scam-type trend, avg time-to-report, completion rate
New user group (elderly, rural)	"Help me" mode: one question per screen, voice, call-a-family-member share link
Security of the app	Rate limit `/api/extract`, file type/size checks, no PII in logs, CSP headers
Different fraud channel (card, net banking)	Method selector already in S3; template wording switches by method
Business twist (pricing, competitor)	Business teammate updates slide; tech adds a feature flag or dashboard tile

Always: commit, tag `challenge-N`, and show the change at the next jury visit.

---

21. Pitch & live demo script (3 minutes)

Time	Speaker	Say	Show
0:00–0:25	Business	"Ravi lost ₹18,000 to a fake KYC call. For six days he did nothing — he didn't know what to do."	Slide 1
0:25–0:55	Business	"51% of victims never file. Odisha recovered 2.7%. And from Jan 2027, RBI compensates only if you report to 1930 and your bank within 5 days."	Slide 2 (3 big numbers)
0:55–1:55	Tech	Live on Stage: open app → "No, it's over" → sample screenshot → fields appear → scam type 92% → countdown red, "Eligible ₹15,300" → step 1 (paste ack) → step 2 → step 3 → **dashboard row turns green**	Stage page
1:55–2:25	Business	"Banks pay for this: ₹6 lakh/year per bank. Insurers pay per claim. Victims never pay. Your bank sends you Reclaim."	Slides 5–6
2:25–3:00	Both	"Privacy-first, no OTPs, no money handling, built on official channels. With Reclaim, Ravi gets ₹15,300 back. Without it, ₹0."	Slide 7

Deck (7 slides): 1 Hook · 2 Problem numbers · 3 Solution (3 steps + countdown visual) · 4 Live demo · 5 Business model · 6 Go-to-market + competition (portal/1930 = destination, guides/blogs = static, consent tools = different problem) · 7 Impact + ask.

Demo hygiene: browser zoom 110%, notifications off, Stage pre-loaded, `Reset demo` pressed just before, backup video open in another tab, phone hotspot on.

Q&A prep

Q	A
Isn't this just a form-filler?	It reads evidence with AI, applies RBI eligibility rules, sequences three official channels, tracks legal deadlines, and gives banks a live claims view. Paperwork is only the output.
Why not just the portal / 1930?	They are the destinations. Most victims never reach all three, in order, on time — and the bank report isn't on the portal.
Privacy?	Victim-shared evidence only, masked fields, delete on request, no OTP/login. Production runs on the bank's paid/self-hosted model so data never leaves them.
OCR wrong?	User confirms every field; manual entry always available.
Is the rule real?	RBI Third Amendment Directions 2026, transactions from 1 Jan 2027 (we've read the notification).
Why would a bank pay?	Fewer incomplete claims, faster resolution within 45 days, compliance trail, fraud-trend insight.
Legal risk?	We never file or promise refunds; we prepare and remind. Final eligibility is the bank's decision.

22. Pre-demo test checklist (run before 09:00 and before 11:00)

- ☐ `/api/health` OK on the demo machine; `DEMO_NOW` set; DB reset + seeded.
- ☐ GPay sample → fields exactly: ₹18,000 · UTR 701245678912 · 12 Jan 2027 7:42 PM · Indian Bank · kyc.update@ybl.
- ☐ Scam type shows "Fake KYC update" ≥ 85%.
- ☐ Countdown shows ~4d 23h and ticks; pill "Eligible for up to ₹15,300".
- ☐ Step 1 accepts `31012270045871`; Step 2 copy works, ≥ 200 chars; Step 3 mailto opens.
- ☐ Done screen shows ₹15,300; PDF downloads and ₹ renders.
- ☐ Dashboard: Ravi's row appears on top, highlights, turns fully green after Step 3.
- ☐ Turn Wi-Fi off → sample still extracts (cache) → whole flow still works locally.
- ☐ Unset `GEMINI_API_KEY` → sample still works (cache), unknown image → manual entry message.
- ☐ 360px mobile width: no overflow. Tamil toggle works on S1/S2/S4/S8.
- ☐ Backup video plays.

23. Risk register

Risk	Mitigation
LLM quota/latency	Cache-first; 12 s timeout; manual path
Venue Wi-Fi	Run local; hotspot; ngrok optional
Countdown wrong (timezone)	All times stored UTC, displayed Asia/Kolkata; demo clock tested
Real data exposure	Only fake samples; no logging of images/text; `.env` git-ignored
Jury: "already built?"	All code written during event; `PRE_EXISTING.md` empty or lists libraries only; frequent commits
Legal accuracy	Use only Section 2 facts; label estimates; "bank decides eligibility" note
Scope creep	Section 3.2 is out of scope; feature freeze 08:00

24. Conventions for AI agents working on this repo

- Read Sections 0, 3, 18 first; pick the lowest unfinished task in the current timeline block.
- Do not change rules constants (7.2) or document templates (8) without the owner's OK.
- Keep functions small and pure where possible (eligibility, docs_gen, validate) and add tests.
- Frontend: TypeScript strict, functional components, Tailwind classes, no UI library beyond lucide-react. All user-facing strings through `t()`.
- Backend: type hints, Pydantic models for every request/response, no blocking calls without timeout.
- Never add features that ask for OTP/PIN/password or that submit to government/bank systems.
- After finishing a task: run tests, update its checkbox in Section 18, commit with a clear message.